

TechCorp Monitoring & Alerting Guide

System Observability, Metrics & Escalation Procedures

Version: v2.3

Document: TC-MON-006

TechCorp Internal

1. System Monitoring Overview

TechCorp uses a unified observability stack built on the Grafana ecosystem. All services emit metrics to Prometheus, traces to Tempo, and logs to Loki. Dashboards and alerts are centrally managed in Grafana Cloud.

Metrics	Prometheus + Grafana Cloud — 15s scrape interval
Tracing	OpenTelemetry SDK + Grafana Tempo — sampled at 10%
Logging	Structured JSON → Fluent Bit → Grafana Loki
Alerting	Grafana Alerting → PagerDuty → Slack
Uptime	Grafana Synthetic Monitoring — probes from 5 regions every 60s
SLO Tracking	Grafana SLO plugin — burn rate alerts for error budget

2. Core Metrics

Metric Name	Type	Description	Labels
http_requests_total	Counter	Total HTTP requests	method, status, route
http_latency_seconds	Histogram	Request latency distribution	method, route, percentile
llm_tokens_used_total	Counter	LLM tokens consumed	model, tenant, type
llm_latency_seconds	Histogram	LLM call latency	model, provider
rag_retrieval_latency	Histogram	Vector search latency	collection, k
queue_depth	Gauge	Jobs pending in processing queue	queue_name
active_connections	Gauge	Active DB connections	db, pool
error_rate	Gauge	Error rate (5xx/total)	service, route

3. Latency Thresholds

API P50	< 120ms — warning at 150ms, critical at 200ms
API P95	< 400ms — warning at 500ms, critical at 800ms
API P99	< 1,200ms — warning at 1.5s, critical at 2.5s
LLM P95	< 2,500ms — provider-dependent; alert on > 5s
RAG Retrieval P95	< 100ms — alert on > 200ms
DB Query P95	< 50ms — warning at 100ms, critical at 300ms
Ingestion Job	< 60s per document — alert on > 120s

4. CPU Usage Thresholds

Warning	70% sustained for 5 minutes — notification only
Critical	85% sustained for 2 minutes — page on-call SRE
Auto-scale Trigger	60% for 3 minutes — scale out by 2 instances
Scale-in Trigger	30% for 15 minutes — scale in by 1 instance
Hard Limit	95% — circuit breaker activates, new requests rejected

5. Memory Usage

Warning	80% heap usage sustained 5 min — investigate for memory leaks
Critical	90% — page SRE; prepare to restart service
OOM Kill	Auto-restart enabled; PagerDuty alert + Slack notification
DB Shared Buffers	Monitor hit ratio > 95%; alert if < 90%
Redis Memory	Alert at 80% maxmemory — triggers eviction policy review

6. Error Rates

Error Budget	99.9% SLO = 43.8 minutes downtime/month; 0.1% error budget
Warning	Error rate > 0.5% over 5-minute window
Critical	Error rate > 1% over 2-minute window — page SRE immediately
LLM 5xx Rate	Alert if provider error rate > 3% in 1 minute — activate fallback
4xx Spike	Alert on 50% increase in 4xx over 10-minute baseline
Burn Rate Alert	Fast burn: > 14.4x budget burn rate (1h window)

7. PagerDuty Alert Rules

Alert Name	Severity	Route	Auto-Resolve
API P95 Critical	High	sre-oncall	Yes
Error Rate Critical	High	sre-oncall	Yes
CPU Critical	High	sre-oncall	Yes
DB Connection Exhausted	Critical	sre-oncall + DBA	No
LLM Provider Failure	High	ai-team	Yes
Security Incident	Critical	security-oncall	No
Data Pipeline Stalled	Medium	data-eng	Yes
Disk Usage > 85%	Medium	sre-oncall	No

8. Escalation Flow

Alert Fires --> PagerDuty notifies on-call engineer (SMS + app push) T+5min: No acknowledgment --> escalate to secondary on-call T+15min: No acknowledgment --> notify Engineering Manager T+30min: Unresolved Sev1 --> escalate to VP Engineering + CTO T+60min: Customer-impacting --> activate Customer Communications runbook T+2h: Major incident declared --> War room bridge opened